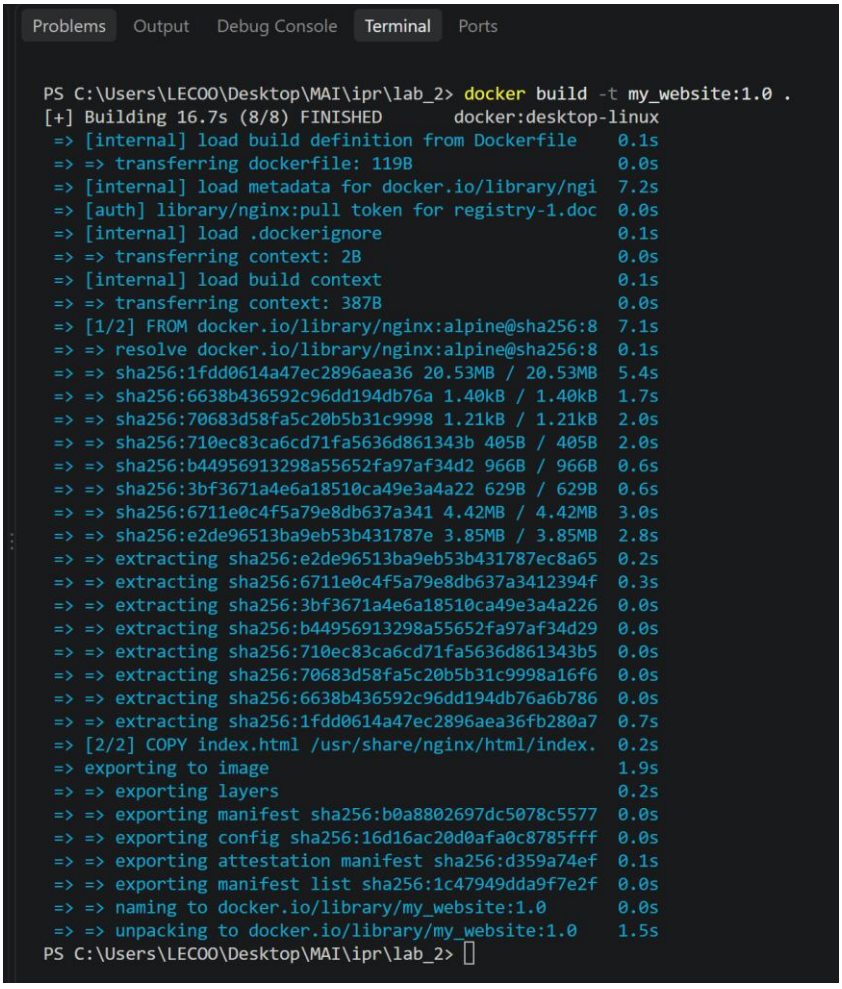
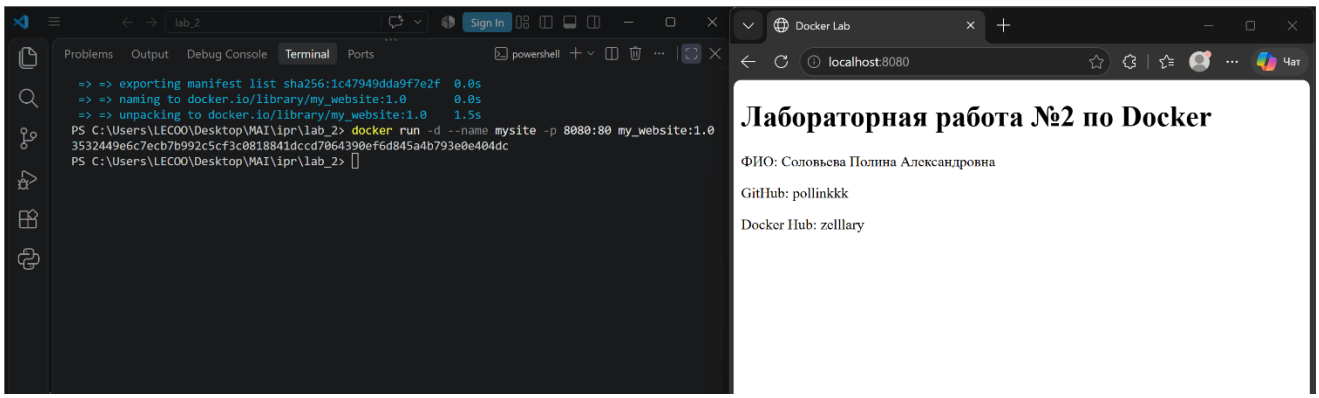
Лабораторная работа №2 по докеру

1.1. С помощью команды docker build -t my_website:1.0 . соберем собственный Docker-образ, который назовем «my_website:1.0» на основе nginx:alpine



Сборка завершилась успешно

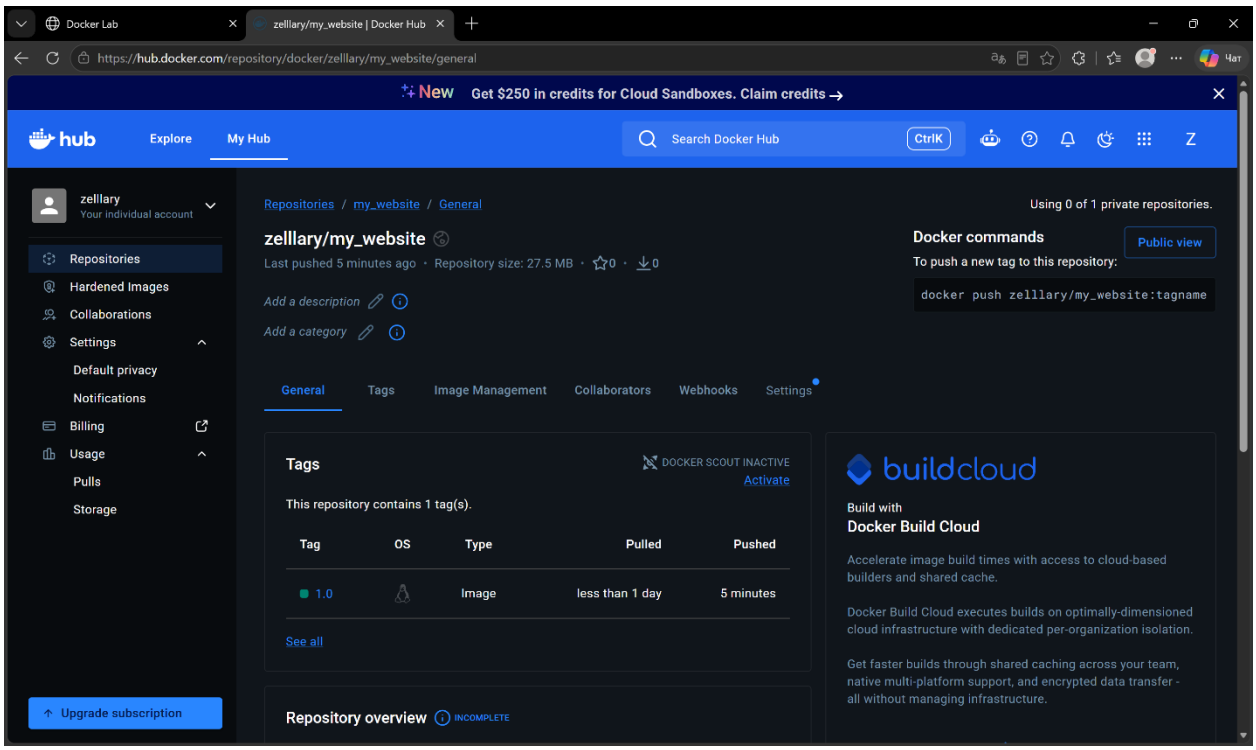
1.2. С помощью команды docker run -d --name mysite -p 8080:80 my_website:1.0 создадим и запустим контейнер из собственного образа. Проверим, что все работает: откроем страницу http://localhost:8080 в браузере. Все открывается и работает



1.3. С помощью команды docker push zelllary/my_website:1.0 загрузим собственный образ в Docker Hub. Загрузка прошла успешно

```
PS C:\Users\LECOO\Desktop\MAI\ipr\lab_2> docker push zelllary/my_website:1.0
The push refers to repository [docker.io/zelllary/my_website]
44136fa355b3: Pushed
e2de96513ba9: Pushed
6711e0c4f5a7: Pushed
6f5dfe1607f7: Pushed
3bf3671a4e6a: Pushed
b44956913298: Pushed
710ec83ca6cd: Pushed
70683d58fa5c: Pushed
6638b436592c: Pushed
1fdd0614a47e: Pushed
026c06ecf015: Pushed
1.0: digest: sha256:1c47949dda9f7e2f91d730279bf417da4eda7a0f96271622f737cd60c4bbd9db
size: 856
PS C:\Users\LECOO\Desktop\MAI\ipr\lab_2>
```

1.4 В браузере откроем Docker Hub и убедимся, что образ с тегом 1.0 загружен

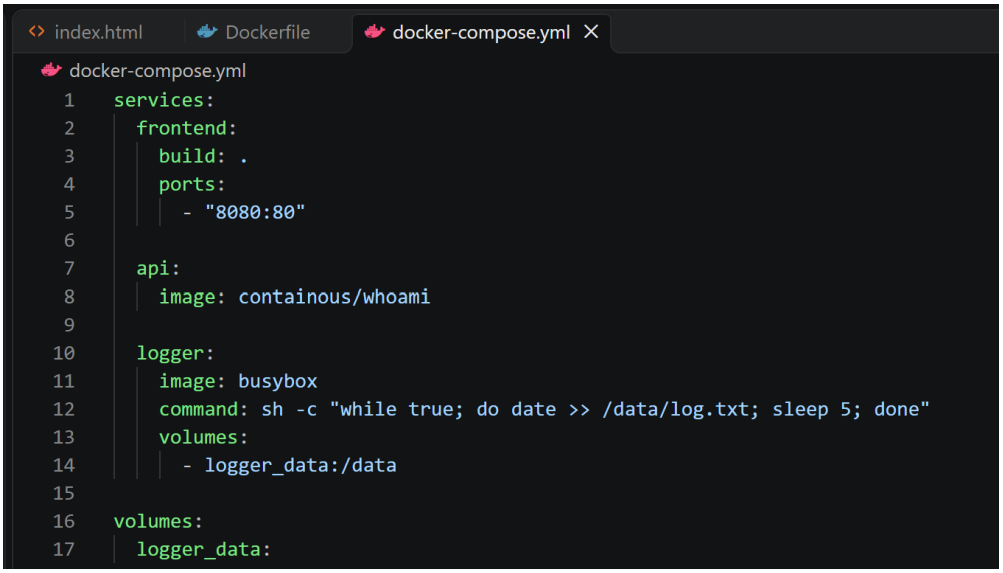


1.5 С помощью команд docker rmi и docker pull (docker rmi zelllary/my_website:1.0 && docker pull zelllary/my_website:1.0). Удалим локальный тег образа, а потом получим образ из Docker Hub. Все выполнилось успешно:

```
C:\Users\LECOO\Desktop\MAI\ipr\lab_2> docker rmi zelllary/my_website:1.0 && docker pull zelllary/my_website:1.0
Untagged: zelllary/my_website:1.0
1.0: Pulling from zelllary/my_website
Digest: sha256:1c47949dda9f7e2f91d730279bf417da4eda7a0f96271622f737cd60c4bbd9db
Status: Downloaded newer image for zelllary/my_website:1.0
docker.io/zelllary/my_website:1.0

C:\Users\LECOO\Desktop\MAI\ipr\lab_2>
```

2.1 Создадим файл docker-compose.yml, в котором настроим 3 сервиса: frontend, api и logger. Для первого укажем сборку из собственного Dockerfile и порты, для api просто укажем готовый образ, а для logger настроим запись времени каждые 5 секунд в файл /data/log.txt с использованием именованного тома logger_data



2.2. С помощью команды docker compose up -d запустим три сервиса: frontend, api и logger, а командой docker compose ps проверим, что все контейнеры находятся в состоянии running

```
C:\Users\LECOO\Desktop\MAI\ipr\lab_2>docker compose up -d
[+] up 7/7
 ✓ Image containous/whoami Pulled                                  9.7s
 ✓ Image busybox           Pulled                                  8.7s
[+] Building 4.3s (10/10) FINISHED
 => [internal] load local bake definitions                         0.0s
 => => reading from stdin 541B                                     0.0s
 => [internal] load build definition from Dockerfile               0.0s
 => => transferring dockerfile: 119B                               0.0s
 => [internal] load metadata for docker.io/library/nginx:alpine    3.4s
 => [auth] library/nginx:pull token for registry-1.docker.io       0.0s
 => [internal] load .dockerignore                                  0.0s
 => => transferring context: 2B                                    0.0s
 => [internal] load build context                                  0.0s
 => => transferring context: 32B                                   0.0s
 => [1/2] FROM docker.io/library/nginx:alpine@sha256:892bb21524    0.0s
 => => resolve docker.io/library/nginx:alpine@sha256:892bb21524    0.0s
 => CACHED [2/2] COPY index.html /usr/share/nginx/html/index.ht    0.0s
 => exporting to image                                             0.2s
 => => exporting layers                                            0.0s
 => => exporting manifest sha256:78da0372b2d2ef4e175745095e0a3f    0.0s
 => => exporting config sha256:e511bbc1451e2ece6e663de99756a4b0    0.0s
 => => exporting attestation manifest sha256:937b9e198a8878ca08    0.0s
 => => exporting manifest list sha256:63f335e7df6ce605b7d1c3370    0.0s
 => => naming to docker.io/library/lab_2-frontend:latest           0.0s
 => => unpacking to docker.io/library/lab_2-frontend:latest        0.0s
 => resolving provenance for metadata file                         0.0s
[+] up 13/13
 ✓ Image containous/whoami    Pulled                               9.7s
 ✓ Image busybox              Pulled                               8.7s
 ✓ Image lab_2-frontend       Built                                4.6s
 ✓ Volume lab_2_logger_data   Created                              0.0s
 ✓ Network lab_2_default      Created                              0.1s
 ✓ Container lab_2-frontend-1 Started                              1.2s
 ✓ Container lab_2-logger-1   Started                              0.5s
 ✓ Container lab_2-api-1      Started                              0.8s

C:\Users\LECOO\Desktop\MAI\ipr\lab_2> docker compose ps
NAME              IMAGE              COMMAND                  SERVICE    CREATED         STATUS         PORTS
lab_2-api-1       containous/whoami  "/whoami"                api        9 minutes ago   Up 9 minutes   80/tcp
lab_2-frontend-1  lab_2-frontend     "/docker-entrypoint.…"   frontend   9 minutes ago   Up 9 minutes   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
lab_2-logger-1    busybox            "sh -c 'while true; …"   logger     9 minutes ago   Up 9 minutes
```

2.3. С помощью команды docker compose exec frontend sh откроем терминал контейнера frontend и выполним запрос curl http://api, получили ответ от API, что подтверждает работу сети Docker и обращение к сервису по его имени

```
C:\Users\LECOO\Desktop\MAI\ipr\lab_2> docker compose exec frontend sh
/ # curl http://api
Hostname: f3aeaf7b0541
IP: 127.0.0.1
IP: ::1
IP: 172.18.0.3
RemoteAddr: 172.18.0.4:48996
GET / HTTP/1.1
Host: api
User-Agent: curl/8.22.0
Accept: */*

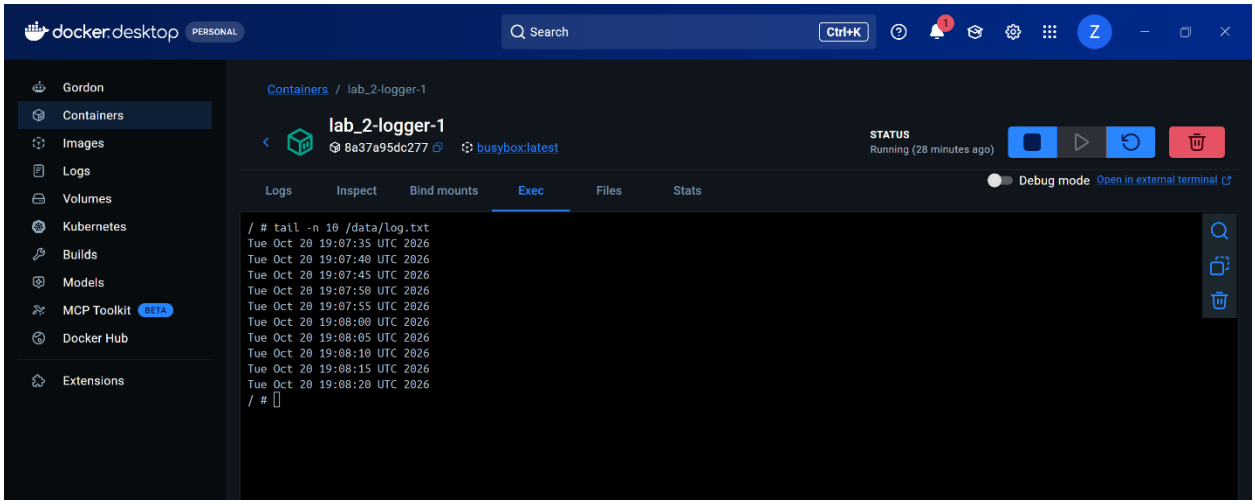
/ #
```

2.4. С помощью команды docker volume ls | grep logger проверим наличие именованного тома lab_2_logger_data, созданного для хранения записей логгера.

```
LECOO@DESKTOP-MT8RPJT MINGW64 ~/Desktop/MAI/ipr/lab_2
$ docker volume ls | grep logger
local     lab_2_logger_data

LECOO@DESKTOP-MT8RPJT MINGW64 ~/Desktop/MAI/ipr/lab_2
$
```

2.5. Откроем терминал контейнера logger в Docker Desktop и с помощью команды tail -n 10 /data/log.txt выведем последние 10 записей и все они с интервалом 5 секунд



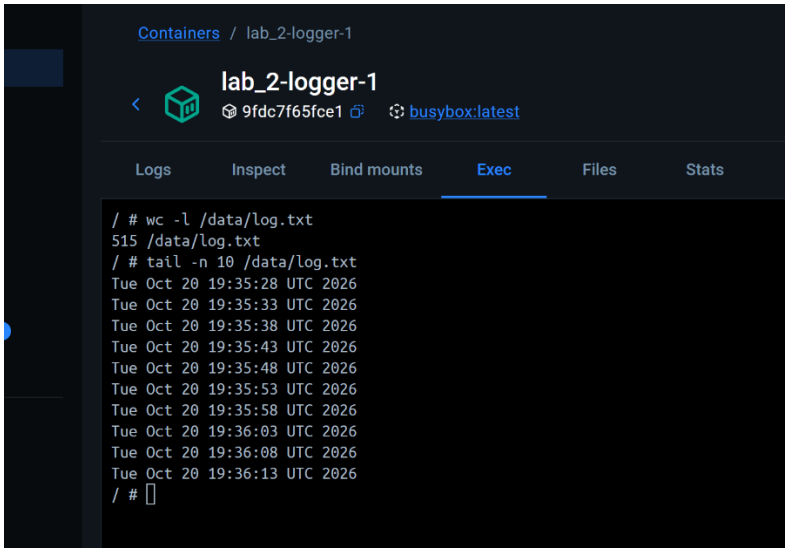
3.1. С помощью команды docker compose down удаляем контейнеры и сеть, сохранив именованный том, а затем командой docker compose up -d повторно создаем и запускаем все три контейнера

```
C:\Users\LECOO\Desktop\MAI\ipr\lab_2> docker compose down
[+] down 4/4
 ✓ Container lab_2-frontend-1 Removed                    0.5s
 ✓ Container lab_2-logger-1   Removed                    3.5s
 ✓ Container lab_2-api-1      Removed                    0.4s
 ✓ Network lab_2_default      Removed                    0.3s

C:\Users\LECOO\Desktop\MAI\ipr\lab_2> docker compose up -d
[+] up 4/4
 ✓ Network lab_2_default      Created                    0.2s
 ✓ Container lab_2-logger-1   Started                    1.2s
 ✓ Container lab_2-frontend-1 Started                    0.9s
 ✓ Container lab_2-api-1      Started                    0.5s

C:\Users\LECOO\Desktop\MAI\ipr\lab_2>
```

3.2. После пересоздания контейнера проверим содержимое файла log.txt. Количество строк увеличилось с 438 до 515 и появились новые записи, что подтверждает, что данные сохранились в именованном томе



3.3. С помощью команды docker compose down -v удалим контейнеры вместе с именованным томом, после повторного запуска в log.txt видим, что старые записи удалились, а файл начал заполняться заново

